

02 Matrices



2.6 Properties of Matrix Multiplication

Chained Multiplication, Associativity, Matrix Decomposition, Commutativity, and Matrix Powers

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills for This Section

1

Chained matrix multiplication: several geometric operations applied to a vector in sequence, taking effect from right to left

3

Associativity: changing where the parentheses go does not change the result, and can be used to optimize the order of computation

5

Commutativity generally fails: even when the product exists, $\mathbf{AB}$ and $\mathbf{BA}$ can carry different geometric meanings

7

Matrix powers: repeatedly multiplying a square matrix by itself represents the same linear transform applied over and over

2

Transpose of a chained product: $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ — reversed order, understood from the row-vector point of view

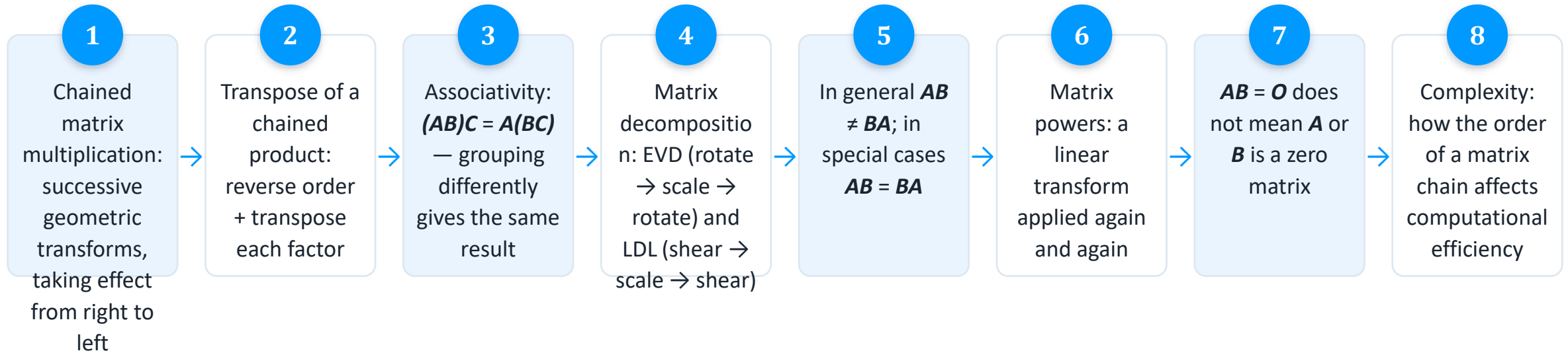
4

Matrix decomposition: breaking a complex matrix into a chain of basic geometric transforms for a more intuitive understanding of its effect

6

Commutativity holds in special cases: the identity matrix, and matrices of the same type — scaling, shear, or rotation

How This Section Unfolds



With the Geometric View, These Properties Become Intuitive

With the geometric view of matrix multiplication in place, the common properties of matrix multiplication become far easier to understand

- In the previous section, we saw that $\mathbf{Ax} = \mathbf{y}$ represents a single geometric transform applied to vector $\mathbf{x}$
- This section builds on that geometric view to understand chained multiplication, associativity, decomposition, commutativity, and matrix powers

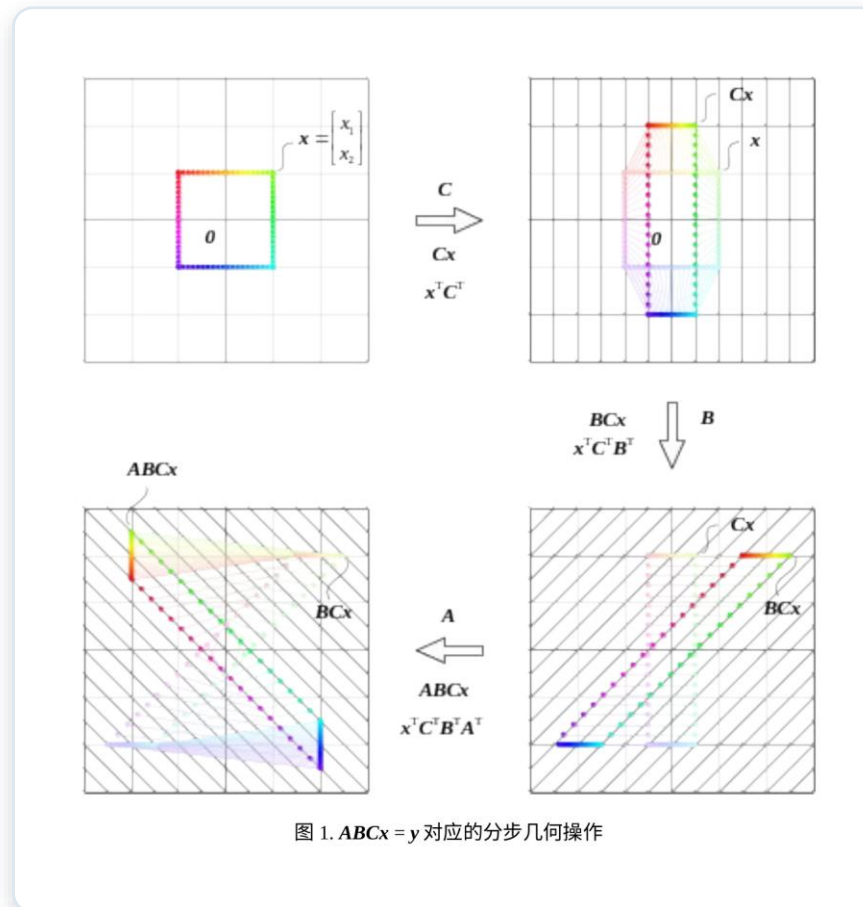
Chained Matrix Multiplication: Successive Geometric Transforms

Multiplying several matrices in sequence corresponds to applying those geometric transforms to a shape one after another

- Take three 2×2 matrices as an example: $\mathbf{A} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ is a 90-degree counterclockwise rotation about the origin
- $\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ is a shear along the x_1 axis; $\mathbf{C} = \begin{bmatrix} 1/2 & 0 \\ 0 & 2 \end{bmatrix}$ shrinks the horizontal axis by half and stretches the vertical axis by 2

Figure 1 · The Step-by-Step Geometric Operations Behind $ABCx = y$

- $ABCx = y$ means column vector x is scaled (C), then sheared (B), then rotated (A), in that order
- Pay close attention to the order: it runs from right to left, that is, $C \rightarrow B \rightarrow A$



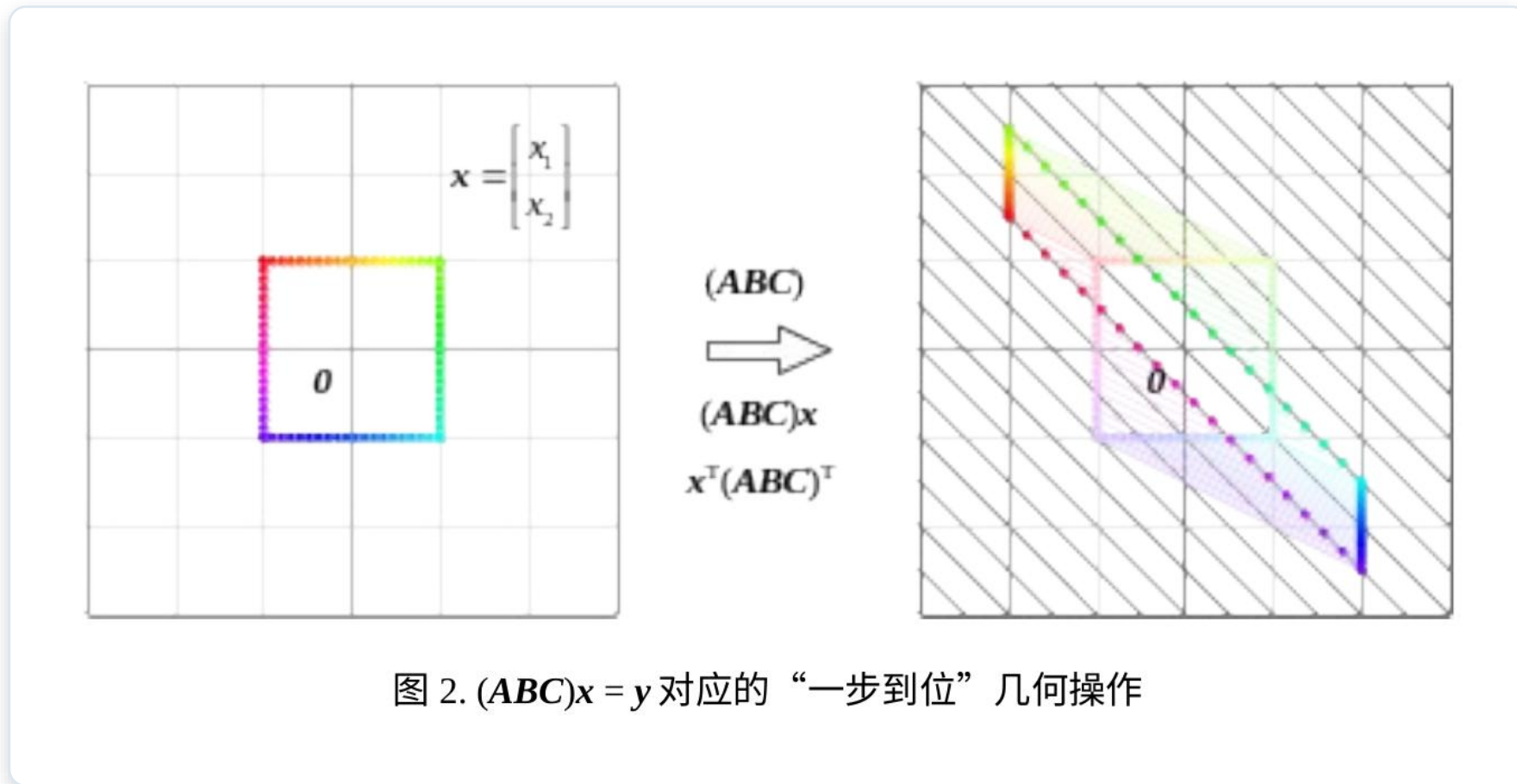
(ABC): Completing the Geometric Transform “All at Once”

We can also compute (ABC) first, and then apply it as a single combined matrix to x

- The product (ABC) represents a composite geometric operation, completing three transforms in one step
- Try computing ABC , CBA , ACB , BAC , and other orderings by hand, then compare the results

Figure 2 · The “All at Once” Geometric Operation Behind $(ABC)x = y$

- Treating (ABC) as a single matrix and applying it once to x gives exactly the same result as Figure 1
- Whether the steps are applied one at a time or combined first, the geometric effect is identical — this is the intuition behind associativity



Transpose of a Chained Matrix Product

$$(AB)^T = B^T A^T, (ABC)^T = C^T B^T A^T, (A_1 A_2 \cdots A_k)^T = A_k^T \cdots A_2^T A_1^T$$

- You don't need to memorize this rule: $\mathbf{x}^T @ \mathbf{C}^T @ \mathbf{B}^T @ \mathbf{A}^T = \mathbf{y}^T$
- Transposing column vector $\mathbf{x}$ gives row vector $\mathbf{x}^T$, a point in the horizontal plane; applying scale ($\mathbf{C}$), shear ($\mathbf{B}$), rotate ($\mathbf{A}$) to the row vector in order still yields Figure 1

Associativity: $(AB)C = A(BC)$

Computing AB first and then multiplying by C , or computing BC first and then left-multiplying by A , gives exactly the same final result

- This property lets us simplify matrix computations, choosing whichever order is more efficient or more convenient for derivation
- Note that the order of the matrices themselves still cannot be changed: matrix multiplication generally does not satisfy commutativity, i.e., $AB \neq BA$

Figure 3 · The Step-by-Step Geometric Operations Behind $(AB)Cx = y$

- (AB) is a “local” composite geometric transform: shear (B) first, then rotate (A)
- x is first scaled (C), then the composite transform (AB) is applied, giving y

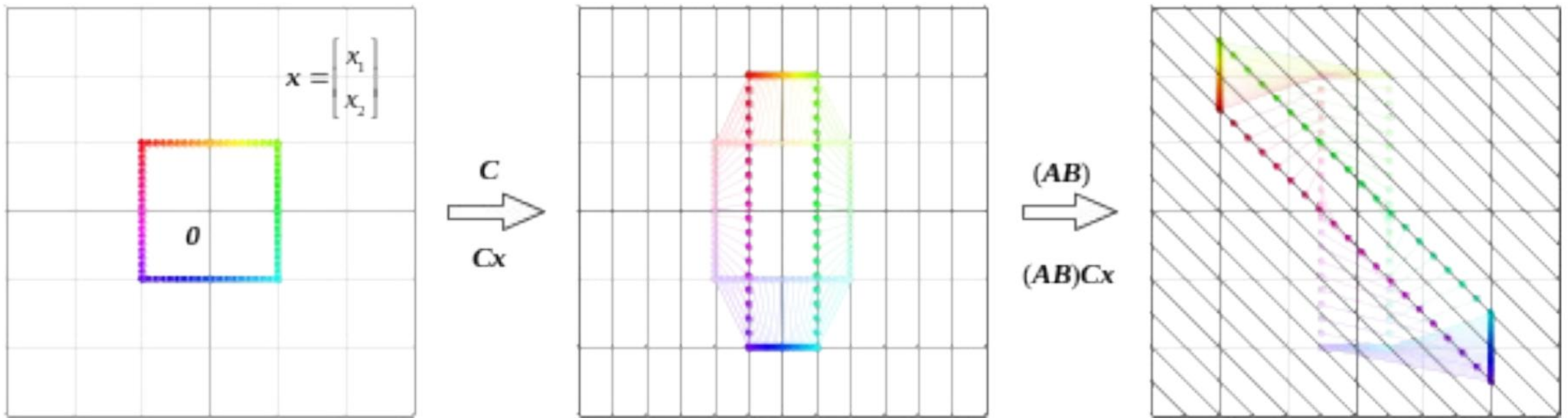


图 3. $(AB)Cx = y$ 对应的分步几何操作

Figure 4 · The Step-by-Step Geometric Operations Behind $A(BC)x = y$

- (BC) is another “local” composite geometric transform: scale (C) first, then shear (B)
- Comparing Figures 1, 2, 3, and 4, the way the operations are expanded or grouped differs, but the final result is always the same

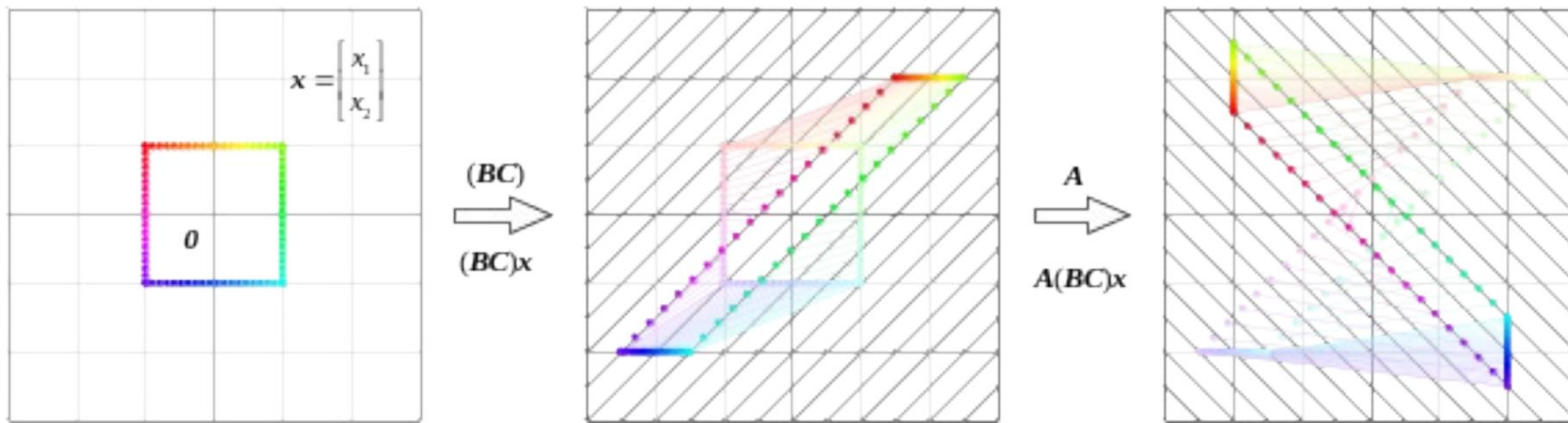


图 4. $A(BC)x = y$ 对应的分步几何操作

Matrix Decomposition

Matrix decomposition breaks a single matrix into a chain of several matrices — the reverse operation of chained multiplication

- Given $\mathbf{A} = \begin{bmatrix} 5/4 & -3/4 \\ -3/4 & 5/4 \end{bmatrix}$, $\mathbf{Ax} = \mathbf{y}$ clearly is not a geometric transform we readily recognize
- Geometrically, a “complicated” operation can be decomposed into a sequence of operations we already understand well

Figure 5 · The Geometric Operation Behind $Ax = y$

- Matrix $A = [5/4, -3/4; -3/4, 5/4]$ turns a square into an irregular quadrilateral
- To understand this transform, we need matrix decomposition to break it into familiar geometric transforms

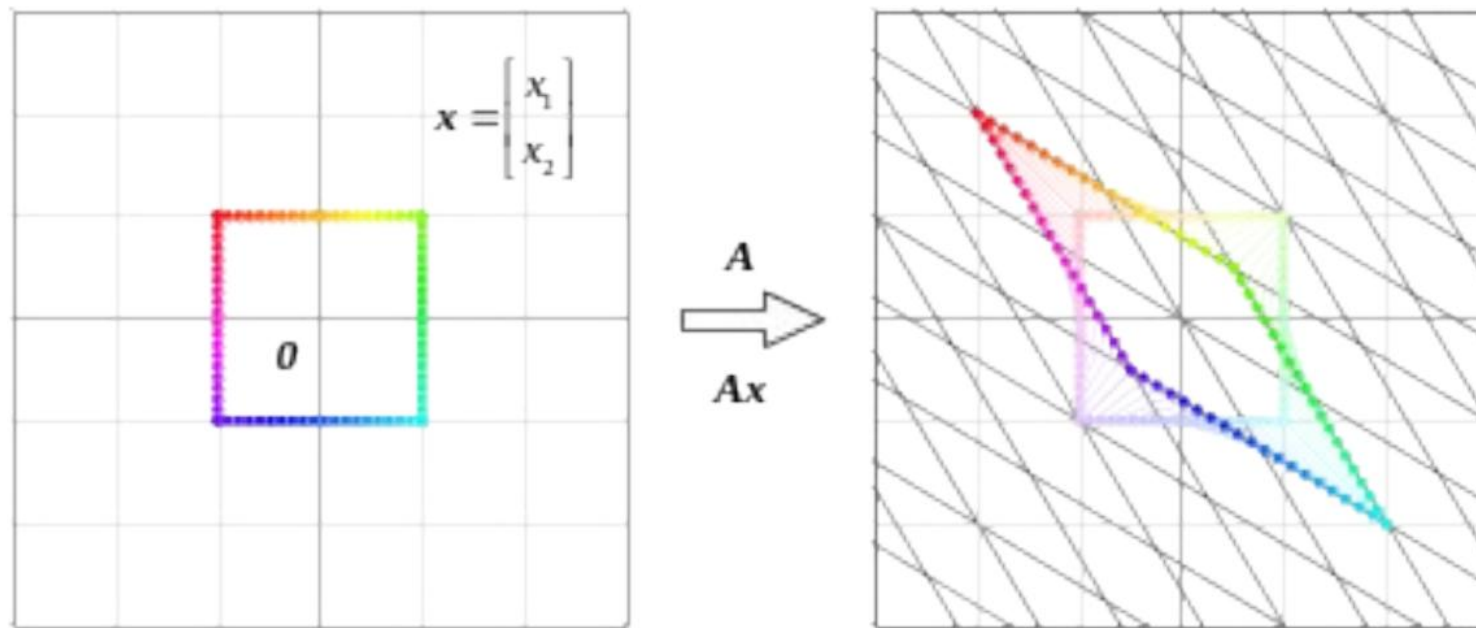


图 5. $Ax = y$ 对应的几何操作

ROTATE → SCALE → ROTATE

Eigenvalue Decomposition (EVD)

$$\mathbf{A} = [\sqrt{2}/2, \sqrt{2}/2; -\sqrt{2}/2, \sqrt{2}/2] @ [2, 0; 0, 1/2] @ [\sqrt{2}/2, -\sqrt{2}/2; \sqrt{2}/2, \sqrt{2}/2]$$

- Matrix $\mathbf{A}$ can be broken into “rotate → scale → rotate,” with the two rotations pointing in opposite directions
- This decomposition is called Eigenvalue Decomposition (EVD); since $\mathbf{A}$ is symmetric, it is also called spectral decomposition

Figure 6 · Decomposing A into “Rotate → Scale → Rotate”

- Step 1 rotates the square into a diamond-like orientation, step 2 scales along the diagonal directions, step 3 rotates again
- After composing the three steps, we get exactly the result of Ax shown in Figure 5

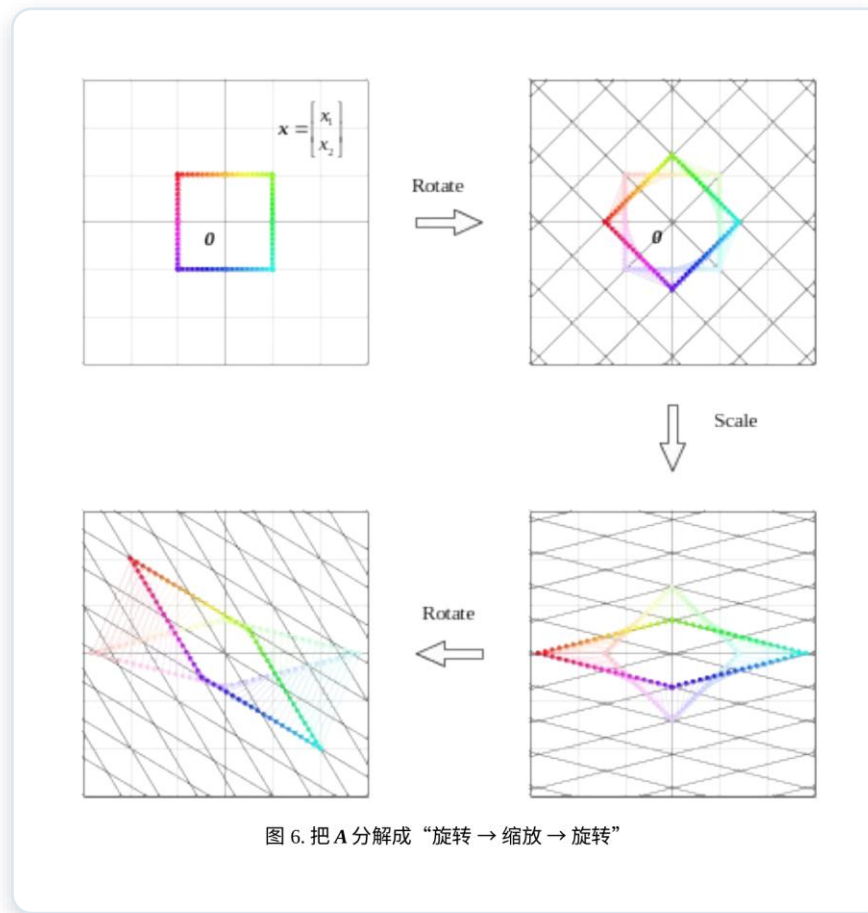


图 6. 把 A 分解成 “旋转 → 缩放 → 旋转”

LDL Decomposition

$$A = [1, 0; -0.6, 1] @ [1.25, 0; 0, 0.8] @ [1, -0.6; 0, 1]$$

- The same matrix **A** can also be broken into “shear → scale → shear,” with one shear along the horizontal axis and the other along the vertical axis
- This is called LDL decomposition; related decompositions include LU decomposition and Cholesky decomposition, which are introduced later in this book

Figure 7 · Decomposing A into “Shear $\rightarrow$ Scale $\rightarrow$ Shear”

- Step 1 shears along the horizontal axis, step 2 scales along the horizontal and vertical axes separately, step 3 shears again along the vertical axis
- The same matrix A admits multiple decompositions, and each one offers its own geometric interpretation

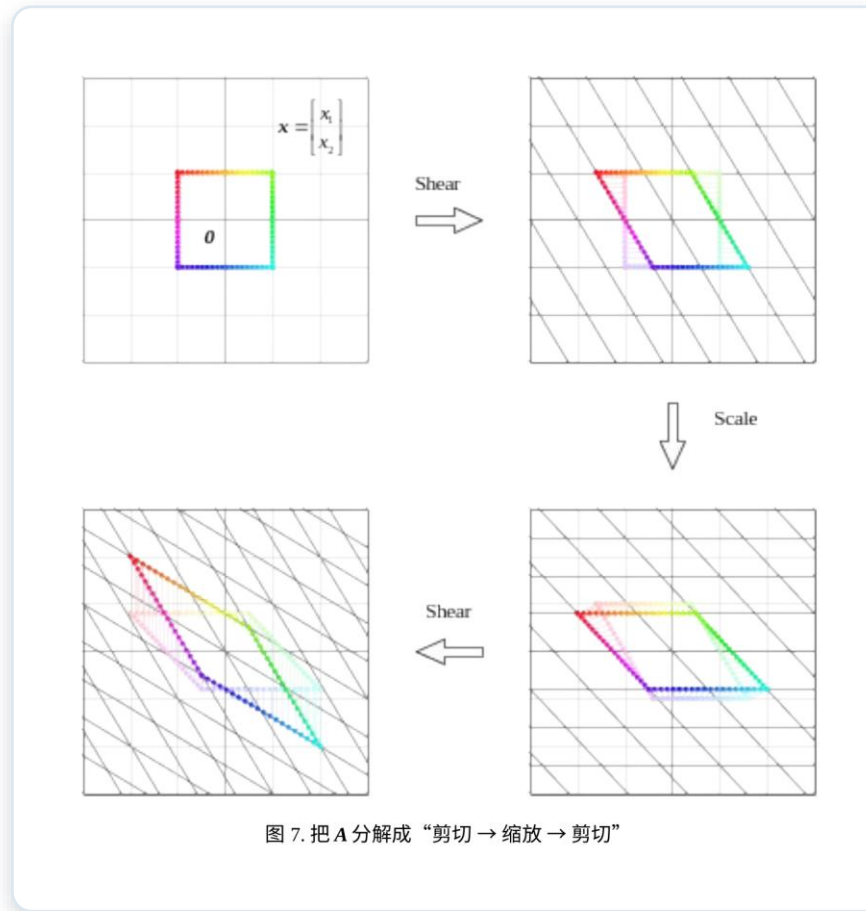


图 7. 把 A 分解成 “剪切 $\rightarrow$ 缩放 $\rightarrow$ 剪切”

In General, AB Does Not Equal BA

Even when both products AB and BA exist, in general $AB \neq BA$

- AB and BA may even have different shapes; and even when the shapes match, they may represent different geometric transforms
- Given $A = \begin{bmatrix} 2 & 0 \\ 0 & 1/2 \end{bmatrix}$ (scaling) and $B = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ (shear), swapping the order changes the result

Figure 8 · The Step-by-Step Geometric Operations Behind $ABx = y$

- ABx means x is first sheared (B), then scaled (A)
- The square is first sheared into a parallelogram, then compressed along the horizontal axis and stretched along the vertical axis

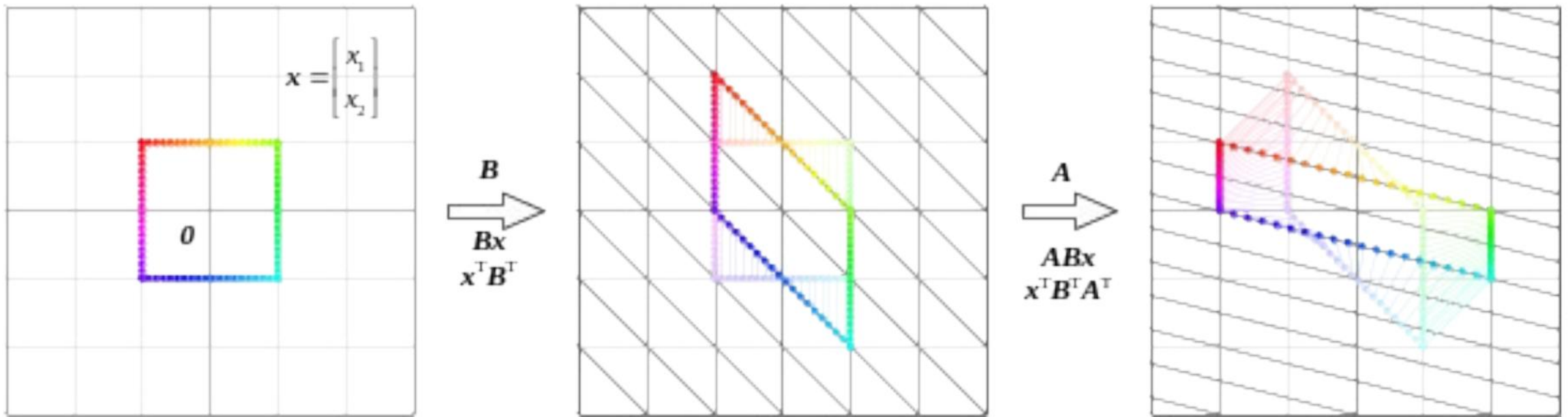


图 8. $ABx = y$ 对应的分步几何操作

Figure 9 · The Step-by-Step Geometric Operations Behind $BAx = y$

- BAx means x is first scaled (A), then sheared (B)
- Comparing Figures 8 and 9, the final results of ABx and BAx are clearly different

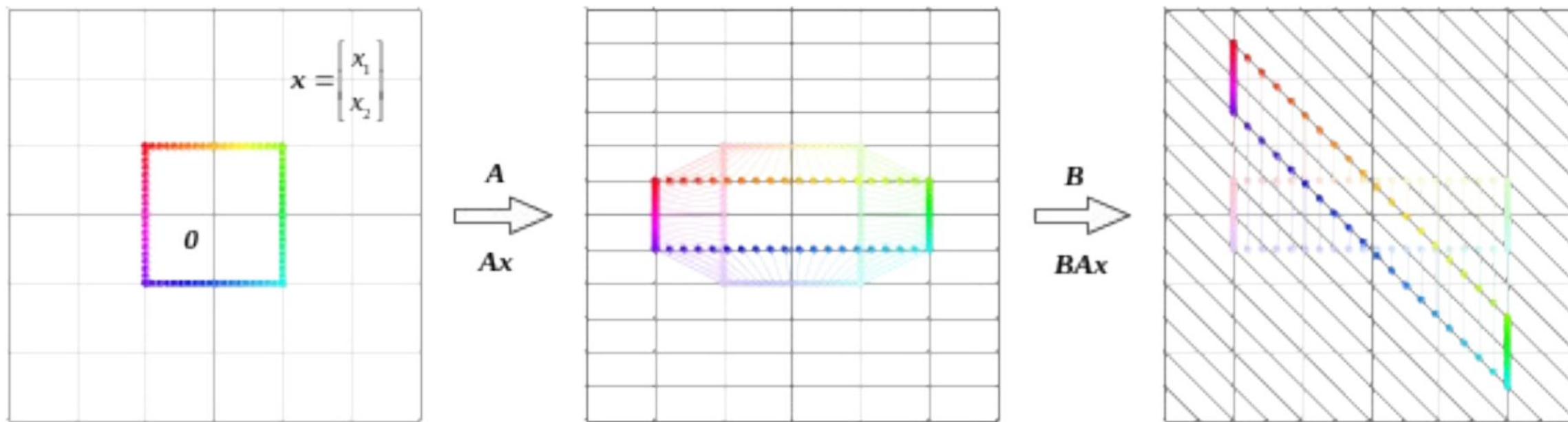


图 9. $BAx = y$ 对应的分步几何操作

In Special Cases, $AB = BA$

If A and B are both identity matrices I of the same shape, then clearly $AB = BA$

- The identity matrix I leaves a shape completely unchanged, so commutativity naturally holds
- Besides the identity matrix, what other special cases satisfy commutativity? The examples below walk through several

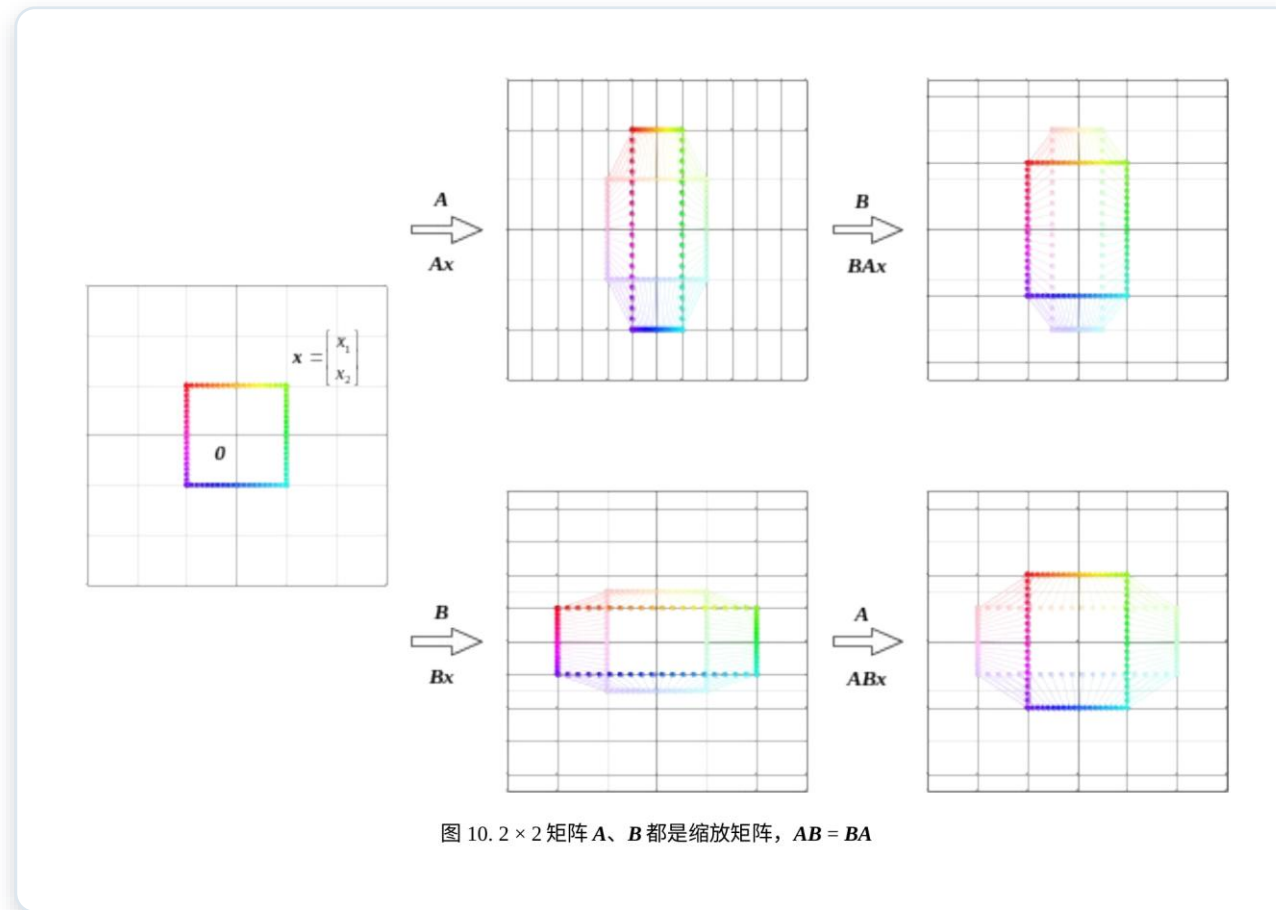
Case 1: A and B Are Both Scaling Matrices

$A = [1/2, 0; 0, 2]$, $B = [2, 0; 0, 2/3]$ — *you can verify $AB = BA$*

- Both scaling matrices are diagonal, and multiplying diagonal entries follows ordinary scalar multiplication, which commutes — so the matrix product commutes too
- Try verifying it yourself: for any two diagonal matrices A and B , does AB always equal BA ?

Figure 10 · 2×2 Matrices A and B Are Both Scaling Matrices, $AB = BA$

- Whether A is applied first and then B , or B first and then A , the final square ends up exactly the same shape
- The two scaling factors along each axis act independently, so the order naturally has no effect on the final result



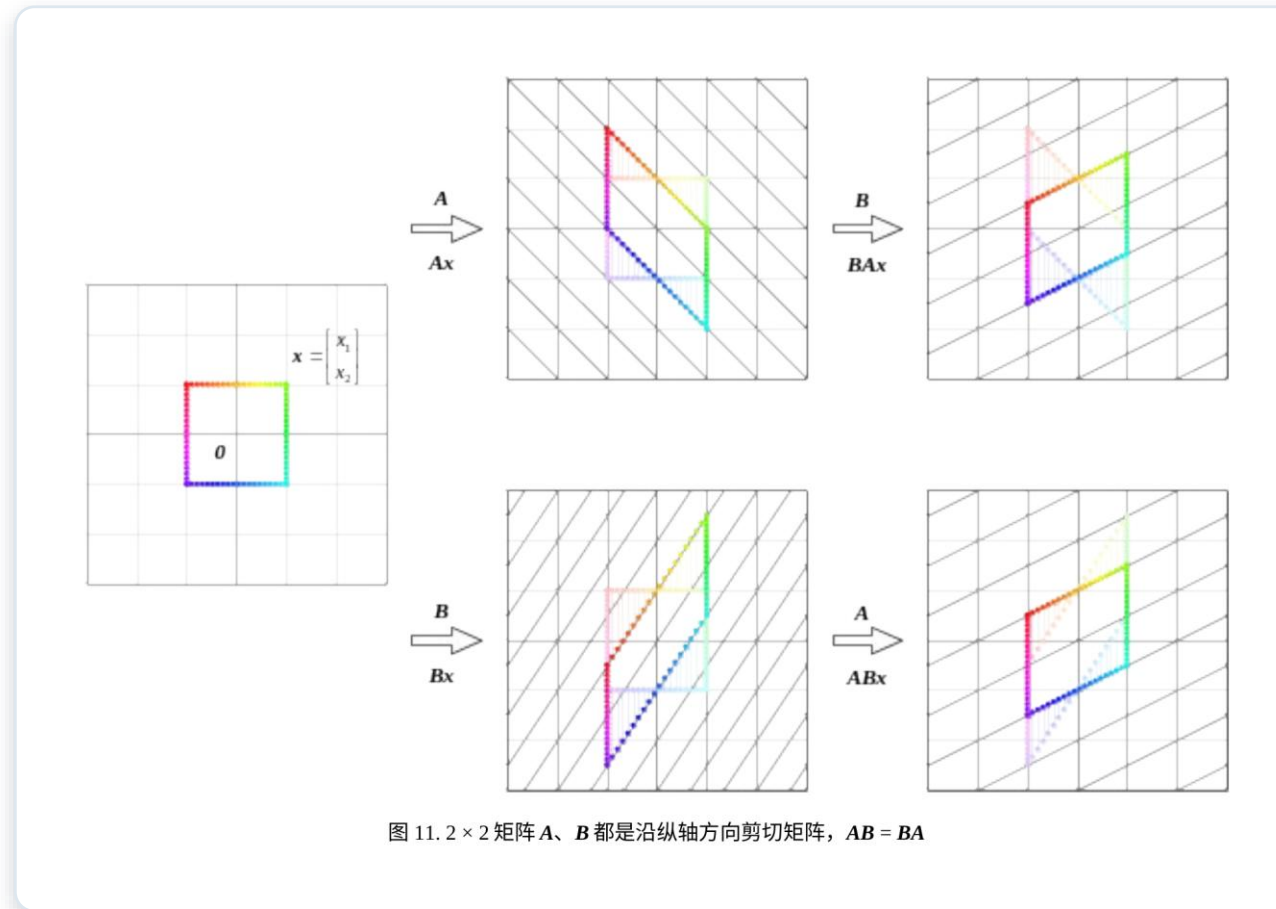
Case 2: A and B Are Both Shears Along the Vertical Axis

$$A = [1, 0; -1, 1], B = [1, 0; 3/2, 1] \text{ — you can verify } AB = BA$$

- When two shear matrices shear along the same direction (the vertical axis), the amounts of shear simply add, so the order does not affect the result
- But if the two shear matrices act along different directions (say, one horizontal and one vertical), commutativity generally fails

Figure 11 · 2×2 Matrices A and B Are Both Shears Along the Vertical Axis, $AB = BA$

- Swapping the order of $A\mathbf{x}$ and $B\mathbf{x}$, the final results $AB\mathbf{x}$ and $BA\mathbf{x}$ are exactly the same
- Two shears in the same direction have the effect of simply adding the shear amounts together, regardless of order



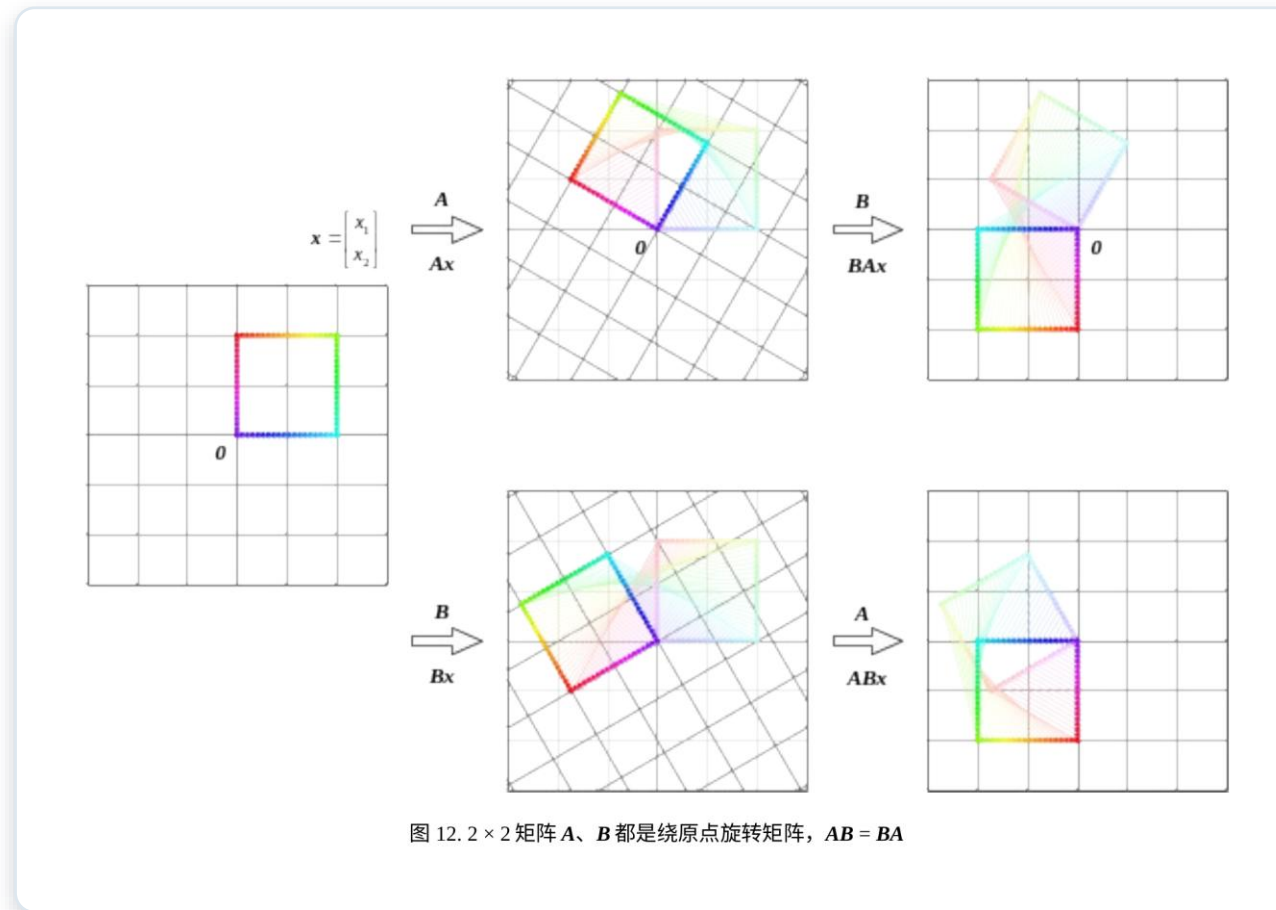
Case 3: A and B Are Both Rotations About the Origin

$A = [1/2, -\sqrt{3}/2; \sqrt{3}/2, 1/2]$, $B = [-1/2, -\sqrt{3}/2; \sqrt{3}/2, -1/2]$ — you can verify $AB = BA$

- Multiplying two rotation matrices is equivalent to adding their rotation angles, and adding angles clearly commutes
- Later in this book we'll revisit these special cases from the standpoint of the underlying geometric mechanisms to explain why they commute

Figure 12 · 2×2 Matrices A and B Are Both Rotations About the Origin, $AB = BA$

- Whether A is applied first then B , or B first then A , the square ends up rotated to exactly the same angle
- The commutativity of rotation matrices ultimately comes from the commutativity of adding rotation angles



Matrix Powers

$$A^0 = I, A^1 = A, A^k = A @ A @ \dots @ A \text{ (} k \text{ copies of } A \text{ multiplied together)}$$

- Matrix powers require $\mathbf{A}$ to be square; k can be a negative integer, in which case $\mathbf{A}$ must be invertible
- Geometrically, a matrix power represents the linear transform $\mathbf{A}$ applied over and over: $\mathbf{A}^k \mathbf{x} = \mathbf{A}(\mathbf{A}^{k-1} \mathbf{x})$

Figure 13 · Powers of a 2×2 Matrix A

- Given $A = [1, -1/2; -1/2, 1]$, repeatedly applying A to vector x ($A^k x$) makes the shape grow larger while continuously rotating
- From Ax to $A^5 x$, each additional multiplication by A scales and rotates the shape a bit more

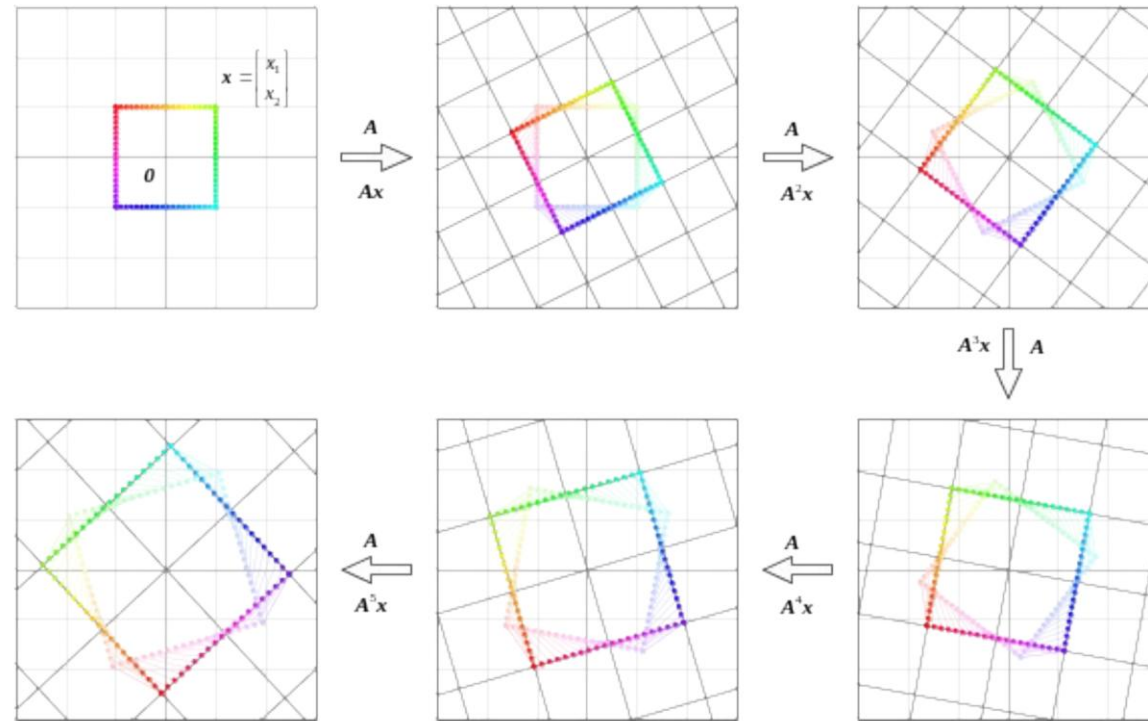


图 13. 2×2 矩阵 A 幂

$$\mathbf{AB} = \mathbf{0}$$

If the product of $\mathbf{A}$ and $\mathbf{B}$ is the zero matrix, i.e., $\mathbf{AB} = \mathbf{0}$, this does not mean $\mathbf{A}$ or $\mathbf{B}$ is a zero matrix, nor does it mean $\mathbf{BA} = \mathbf{0}$

- Given $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$, neither matrix is a zero matrix
- Also, if $\mathbf{A} = \mathbf{B}$, then $\mathbf{AC} = \mathbf{BC}$; but unless $\mathbf{C}$ is invertible, $\mathbf{AC} = \mathbf{BC}$ does not let you conclude $\mathbf{A} = \mathbf{B}$

THE SQUARE “COLLAPSES” TO THE ORIGIN

Figure 14 · 2×2 Matrices A and B Whose Product Is the Zero Matrix, $AB = 0$

- ABx first flattens the square along the vertical axis into a line segment, then compresses it down to the origin
- Even though neither A nor B is a zero matrix, their composition makes all the information “disappear”

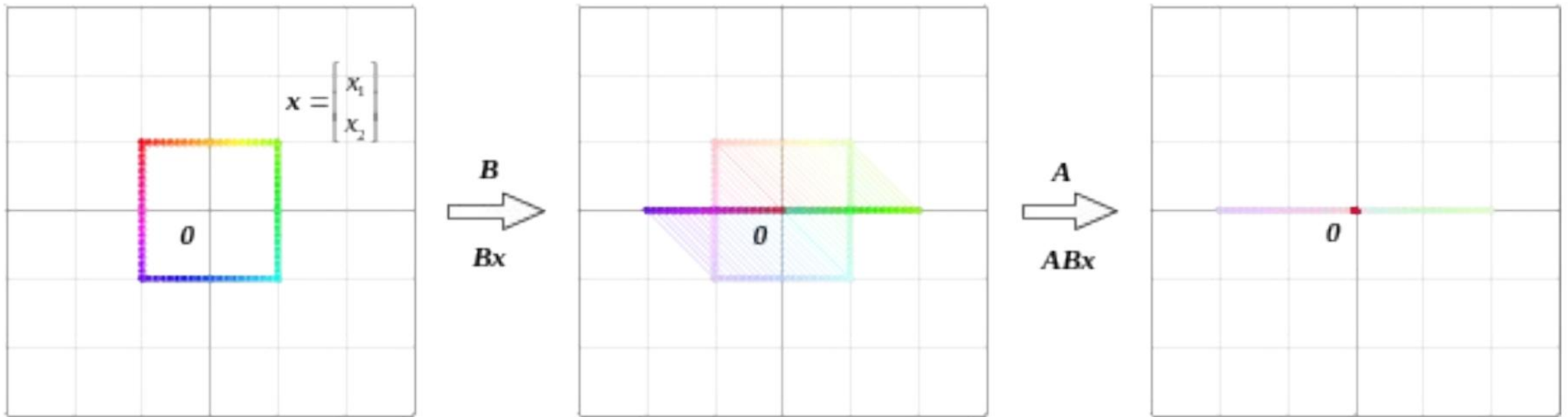


图 14. 2×2 矩阵 A 、 B 矩阵乘积为零矩阵, $AB = 0$

Figure 15 · 2×2 Matrices A and B, Where BA Is Not 0

- BAx does not produce the zero matrix; the square is compressed into a line segment — a “dimension reduction,” not a “collapse”
- This shows that $AB = O$ and $BA = O$ are two independent facts — neither implies the other

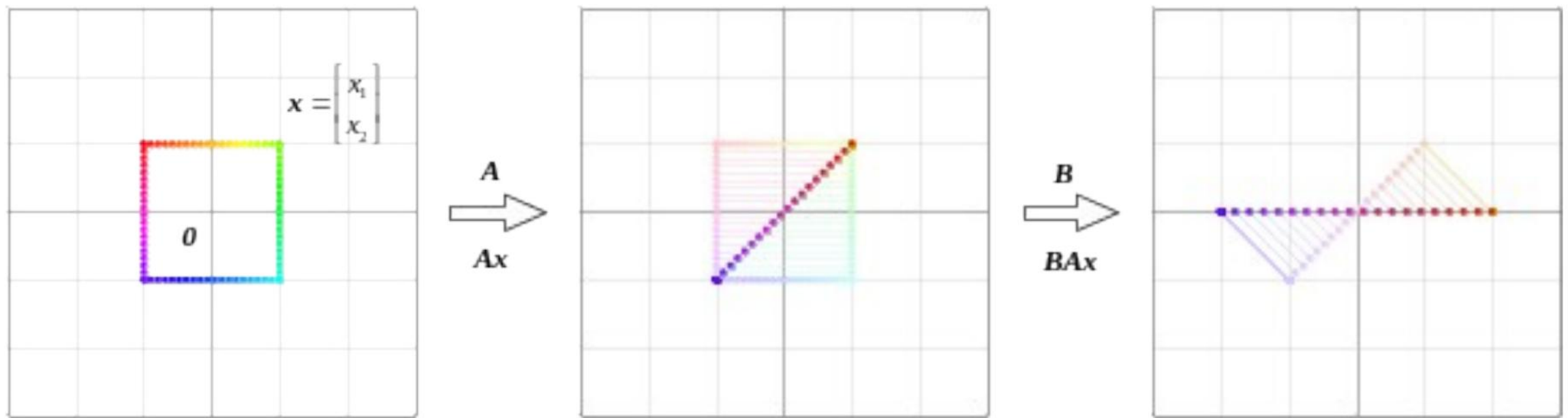


图 15. 2×2 矩阵 A、B 矩阵乘积为零矩阵，BA 不为 O

Complexity

Every intermediate dimension in a chained matrix product cancels out during the computation, leaving only the first and last dimensions, m and n

- The complexity of $\mathbf{C} = \mathbf{AB}$ (with $\mathbf{A}$ of shape $m \times p$ and $\mathbf{B}$ of shape $p \times n$) is $m \times n \times p$, counting only multiplications
- In practice, the order in which a chain is multiplied can significantly affect efficiency, since the total complexity depends on the matrix sizes and the multiplication order

Figure 16 · A Chained Matrix Product

- Five matrices multiplied in sequence; the intermediate dimensions p_1, p_2, p_3, p_4, p_5 between neighboring matrices are cancelled out one by one
- No matter what order you choose to compute in, the shape of the final result matrix is always $m \times n$

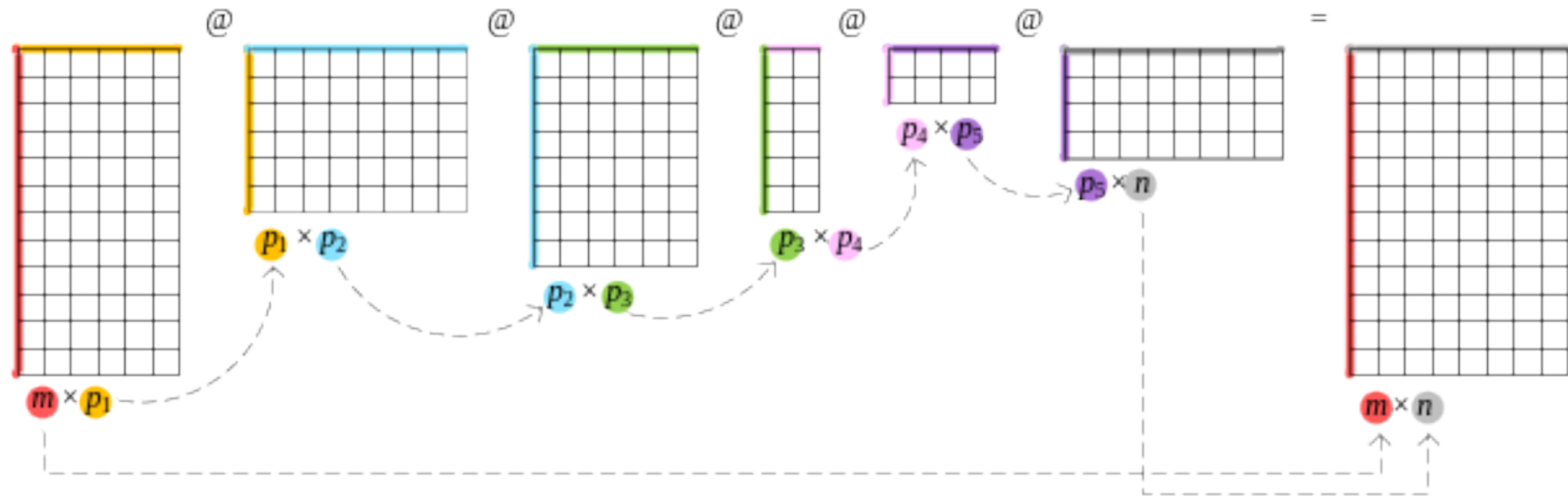


图 16. 矩阵连乘

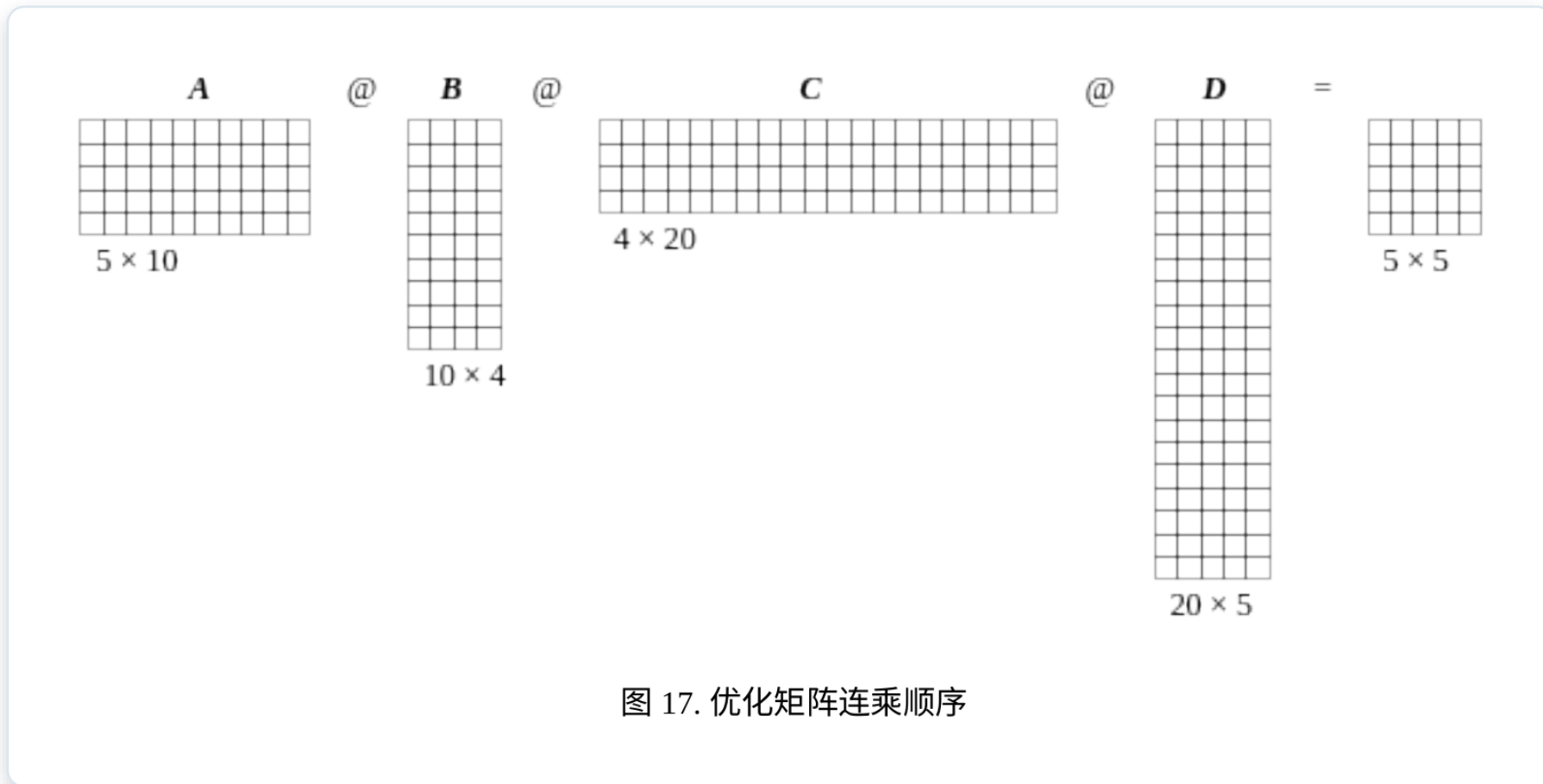
Optimizing the Order of a Matrix Chain

Four matrices chained together: $A(5 \times 10)$, $B(10 \times 4)$, $C(4 \times 20)$, $D(20 \times 5)$

- Order 1: $((A @ B) @ C) @ D$, total complexity = $200 + 400 + 500 = 1100$
- Order 2: $(A @ B) @ (C @ D)$, total complexity = $200 + 400 + 100 = 700$ — about 36% less computation than Order 1

Figure 17 · Optimizing the Order of a Matrix Chain

- A, B, C, D have shapes 5×10 , 10×4 , 4×20 , and 20×5 respectively, and the final product has shape 5×5
- Cancelling out the larger intermediate dimensions first (such as C's 20) can substantially cut later computation and lower the overall complexity



Optimizing Computation Order Is the Foundation of Efficient Matrix Computing

Optimizing computation order, exploiting structural properties, and adopting advanced algorithms together form the theory and practice of efficient matrix computation

- Matrix computation is used everywhere in machine learning and data science; optimizing multiplication order can significantly speed up forward and backward passes in neural networks
- Modern approaches also exploit structural properties such as sparse matrices (skipping zero entries) and block matrices (enabling parallel computation)

Key Terms in This Section

Chained Matrix Multiplication *Several geometric transforms applied one after another, taking effect from right to left*

Associative Law $(AB)C = A(BC)$ — *where the parentheses go doesn't change the result*

Eigenvalue Decomposition *EVD — rotate $\rightarrow$ scale $\rightarrow$ rotate; also called spectral decomposition for symmetric matrices*

Commutative Law *Matrix multiplication generally does not satisfy it, i.e., $AB \neq BA$*

$AB = \mathbf{O}$ *Two nonzero matrices can still multiply to give the zero matrix*

Transpose of a Chain $(ABC)^T = C^T B^T A^T$ — *the order reverses under transpose*

Matrix Decomposition *Breaking a complex matrix into a chain of basic geometric transforms*

LDL Decomposition *shear $\rightarrow$ scale $\rightarrow$ shear; related decompositions include LU and Cholesky*

Power of a Matrix *The k -fold product of square matrix A , representing a linear transform applied repeatedly*

Complexity *The number of multiplications needed for a matrix chain, which depends on the dimensions and the order of multiplication*

GO FURTHER

Companion Code and Further Reading

Further Reading · LA_02_06.ipynb

Code that produces every figure in this section — work through it line by line

Recommended · Book of Matrix Power

A deeper look at the geometry and algebra behind matrix decompositions (EVD, LU, LDL, Cholesky)

Related Content · Eigenvalues and Eigenvectors

A later chapter covers the mathematics behind EVD in depth

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Questions Q1 – Q9

Q1 Use a random positive-integer generator to create four 2×2 matrices A, B, C, D, and write your own Python code to compute every permutation of the chained product (e.g., $A @ B @ D @ C$).

Q3 Matrix A below shears along the horizontal axis, and B shears along the vertical axis: $A = \begin{bmatrix} 1 & 1 & 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 0 & 1 & 1 \end{bmatrix}$. Does $AB = BA$?

Q5 Given 7 different 2×2 matrices A (covering scaling, shear, rotation, reflection, and more), compute A^8 for each and explain geometrically what repeated application of a linear transform does.

Q7 Use Python to verify the LDL decomposition in Figure 7, and compare the intermediate shapes produced by EVD versus LDL.

Q9 Use NumPy to randomly generate two 3×3 matrices and verify whether the transpose formula $(ABC)^T = C^T B^T A^T$ holds.

Q2 Matrix A below scales along the horizontal axis, and B scales along the vertical axis: $A = \begin{bmatrix} 2 & 0 & 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 0 & 0 & 3 \end{bmatrix}$. Does $AB = BA$?

Q4 Given horizontal-shear matrices $A = \begin{bmatrix} 1 & k_1 & 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & k_2 & 0 & 1 \end{bmatrix}$, $C = \begin{bmatrix} 1 & k_3 & 0 & 1 \end{bmatrix}$, $D = \begin{bmatrix} 1 & k_4 & 0 & 1 \end{bmatrix}$, compute the product ABCD. What pattern do you notice?

Q6 Use Python to verify the EVD decomposition in Figure 6: does $A = \begin{bmatrix} 5/4 & -3/4 & -3/4 & 5/4 \end{bmatrix}$ equal the product of the three matrices given? Visualize the three-step decomposition.

Q8 Write a function `commute_check(A, B)` that determines whether two given square matrices satisfy $AB = BA$, and have it print the actual values of AB and BA.

Questions Q10 – Q18

Q10 Use Python to verify that associativity $(AB)C = A(BC)$ holds for any matrices with compatible shapes, including non-square matrices.

Q12 Write a function `matrix_power_visualize(A, k)` that plots x , Ax , A^2x , ..., A^kx in sequence, reproducing the effect seen in Figure 13.

Q14 Given a chain of 5 matrices with shapes of your choosing, write a function `compute_all_costs(shapes)` that enumerates every way to parenthesize the chain and computes the complexity of each.

Q16 Use NumPy to verify that if C is invertible, $AC = BC$ implies $A = B$; then construct a counterexample showing this fails when C is not invertible.

Q18 Write a function `decompose_shear_scale_shear(A)` that attempts to automatically solve for the L and D matrices of the LDL decomposition of a given 2×2 matrix.

Q11 Construct two shear matrices in different directions (one horizontal, one vertical), check whether AB equals BA , and explain why.

Q13 Construct a pair of nonzero matrices A , B such that $AB = O$ but $BA \neq O$, verify it with Python, and explain the geometric reason behind it.

Q15 Time the two multiplication orders in Figure 17 with Python, and check whether the measured runtimes match the complexity analysis.

Q17 Construct a 3×3 rotation matrix and a 3×3 scaling matrix, and verify whether AB still fails to equal BA in the three-dimensional case.

Questions Q19 – Q25

Q19 Use Python to draw a Figure-16-style diagram of a chained matrix product: given the shapes of any number of matrices, automatically label the intermediate dimension cancelled at each step.

Q21 Verify that two matrices that both rotate about the origin commute, and check whether the sum of the two rotation angles equals the angle of the single combined rotation.

Q23 Use Python to compare the runtime of multiplying sparse matrices (e.g., a 100×100 matrix that is mostly zeros) versus dense matrices.

Q25 Combining what you learned in this section, write a small program that takes several matrices and a multiplication order, then computes and prints the complexity of each step along with the running total.

Q20 Given a list of shapes for a matrix chain, implement the classic “matrix chain multiplication” dynamic-programming algorithm, and output the optimal parenthesization and its minimum complexity.

Q22 Construct a 3×3 block-diagonal matrix (with a 2×2 block and a 1×1 block), and verify whether block matrix multiplication is equivalent to multiplying each sub-block separately.

Q24 Given matrix A , write a function `is_diagonalizable_hint(A)` that attempts to judge whether A might admit a Figure-6-style “rotate $\rightarrow$ scale $\rightarrow$ rotate” decomposition (hint: check for symmetry).

10 Prompts to Explore Further (1/2)

1 Explain why the chained product $ABCx = y$ takes effect from right to left, and how this compares to function composition $f(g(h(x)))$.

2 Prove the associative law $(AB)C = A(BC)$, and explain why this property is so important for large-scale matrix computation.

3 Explain the geometric meaning of “rotate $\rightarrow$ scale $\rightarrow$ rotate” in eigenvalue decomposition (EVD), and why the two rotations point in opposite directions for a symmetric matrix.

4 Explain why two shear matrices in the same direction commute, while two shear matrices in different directions generally do not.

5 Explain why $AB = O$ does not imply that A or B is a zero matrix, using the column space / null space of a matrix for intuition.

10 Prompts to Explore Further (2/2)

6

Explain the geometric meaning of the matrix power A^k , and describe the spiral growth pattern that appears when A has both a “rotation component” and a “scaling component”.

7

Explain why the computational complexity of matrix multiplication is $m \times n \times p$, and derive the complexity formulas for two different parenthesizations of a chain.

8

Explain the core idea behind the “matrix chain multiplication” dynamic-programming algorithm, and how it finds the optimal solution among an exponential number of parenthesizations.

9

Give examples of how matrix decompositions (EVD, LU, LDL, Cholesky) are used in practice in machine learning or data compression.

10

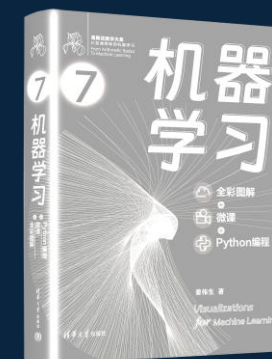
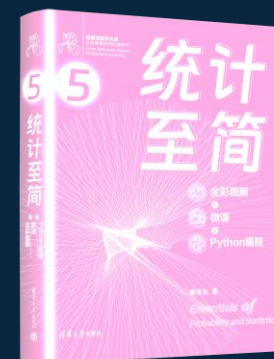
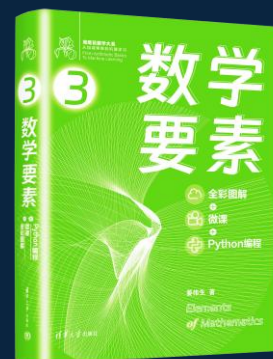
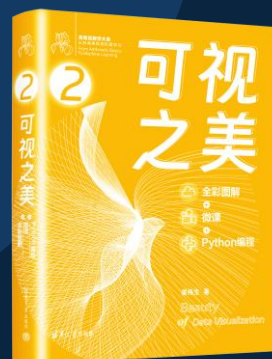
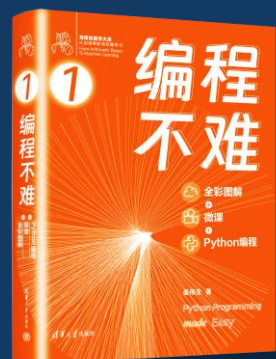
Explain why optimizing the order of matrix multiplication in the forward pass of a neural network can significantly speed up training and inference.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

